

DIA 4

Multiagentes: topologias e orquestração

Cinco topologias, isolamento de contexto, paralelismo e observabilidade

1

Quando dividir, e quando não

2

Supervisor, handoff, rede,
hierárquico, sequencial

3

Context engineering como
orçamento

4

Memória, interface e traço

As oito horas de hoje

1

09:00 – 10:30 • Quando dividir, e as cinco topologias

O aviso que abre o dia, os critérios de decisão e o desenho de cada topologia.

2

10:45 – 12:15 • Implementando, e laboratório

Handoff, supervisor, fábrica de especialistas, limite de ciclo. Depois, teclado.

3

13:15 – 14:45 • Paralelismo, subgrafos e context engineering

Send, reducers, o bug mais difícil do dia, e o orçamento de contexto.

4

15:00 – 16:30 • Memória, interface e laboratório

Memória semântica, Gradio, e a tabela comparativa das duas topologias.

5

16:30 – 17:00 • Observabilidade e fechamento

Quatro métricas, traço em disco e o gancho do dia 5.

01

Antes de dividir

Multiagente não é upgrade automático.

O AVISO

Um sistema com cinco agentes é cinco vezes mais lento, gasta muito mais tokens e falha de formas mais criativas.

A pergunta certa não é como faço multiagente. É: o meu agente único está falhando por qual motivo? Sem essa resposta, dividir apenas espalha o problema por mais lugares e torna o diagnóstico mais difícil.

Motivos que justificam, e os que não

X Não justificam

- Fica mais bonito no diagrama
- Vi num vídeo
- Quero que pareça uma equipe
- Dar nome de cargo a um prompt não cria competência
- Achei que ia ficar mais inteligente

V Justificam

- Prompt de sistema com mais de 60 linhas de regras conflitantes
- Mais de 8 a 10 ferramentas e o modelo escolhe errado
- Fases com critérios de sucesso distintos
- Contexto estoura porque tudo vive na mesma conversa
- Quero um revisor não contaminado pelo autor
- Partes independentes poderiam rodar em paralelo

O que cinco agentes custam de verdade

5x

chamadas de modelo por tarefa

2x

transições que não produzem
trabalho

5x

prompts para manter

1

traço muito mais difícil de ler

Nenhum desses números é motivo para não dividir. Todos são motivo para dividir com evidência, e não por estética. O ganho precisa ser maior que a soma dos quatro, e o único jeito de saber é medir as duas versões.

O teste do agente único

1

Meça a taxa de escolha errada de ferramenta

Vinte execuções, conte quantas vezes o agente escolheu a ferramenta errada. Se for baixa, o problema não é excesso de ferramentas.

2

Meça os tokens de entrada por chamada

Se o contexto não está perto do limite, dividir não vai resolver o que você acha que vai.

3

Corte o prompt pela metade e meça de novo

Metade das regras de um prompt inflado não muda nada. É o critério de poda do dia 1 aplicado aqui.

4

Só então divida

E divida no menor número de agentes que resolve a evidência. Dois costumam bastar; cinco quase nunca são necessários no primeiro corte.

O que dividir NÃO resolve

1 Modelo fraco continua fraco

Cinco instâncias do mesmo modelo pequeno não somam inteligência. Somam custo e pontos de falha.

2 Ferramenta ruim continua ruim

Se a description não diz quando usar, o especialista erra igual. O problema é de interface, não de organização.

3 Falta de critério de pronto continua faltando

Um Crítico sem critérios verificáveis aprova tudo. Você trocou um agente sem feedback por dois.

Decida cada caso

1

**Um agente com 4 ferramentas
erra 2% das escolhas**

Dividir?

2

**O prompt tem regras de redação
e de conferência numérica
misturadas**

Dividir?

3

**Precisa analisar 30 documentos
independentes**

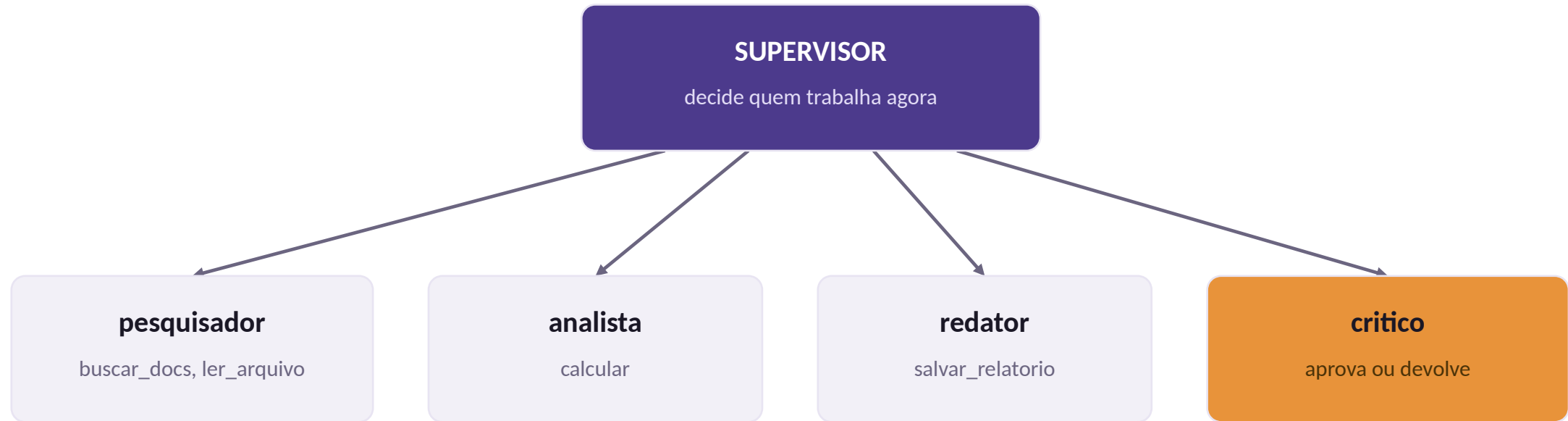
Dividir?

02

As cinco topologias

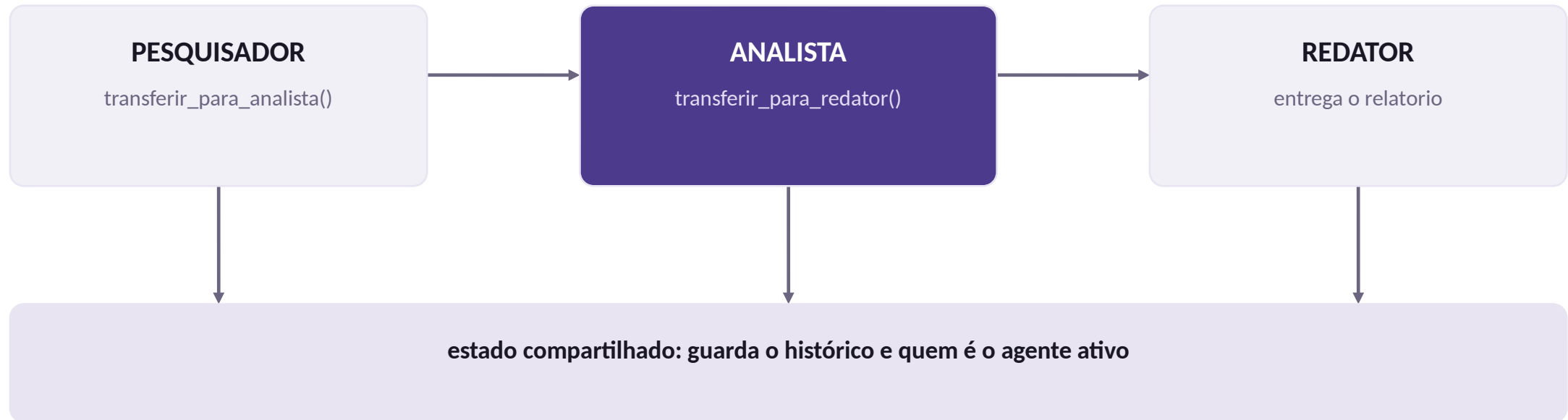
Cinco formas de organizar, com trocas diferentes.

Supervisor: o padrão



Fluxo legível, um único ponto de decisão, fácil de logar e testar. Custo: o supervisor é gargalo e toda transição gasta uma chamada de modelo a mais.

Handoff: sem coordenador



Menos chamadas de modelo, a conversa flui, e quem tem o contexto decide. Custo: ninguém tem visão global e há risco de dois agentes trocando indefinidamente.

Agentes como ferramentas

Um agente inteiro pode ser embrulhado como uma ferramenta do agente principal. O chamador não sabe que do outro lado existe um loop: ele vê um nome, um schema e um resultado.

É a forma mais simples de composição e a que menos exige do LangGraph. O isolamento de contexto vem de graça: o subagente só recebe o argumento da chamada.

O custo é que o principal não vê o raciocínio do subagente, só o resultado. Para depurar, o traço do subagente precisa ir para disco.

```
@tool
def pesquisar_a_fundo(assunto: str) -> str:
    """Pesquisa um assunto e devolve um resumo
    com fontes. Use quando a pergunta exigir
    varias buscas encadeadas.
    """

    r = agente_pesquisador.invoke(
        {"messages": [HumanMessage(assunto)]})
    return r["messages"][-1].content

principal = create_agent(
    model=llm,
    tools=[calcular, pesquisar_a_fundo],
    system_prompt=PROMPT_PRINCIPAL)
```

Rede, hierárquico e sequencial

R Rede

Todos falam com todos. Com n agentes há n vezes n menos 1 transições possíveis. Máxima flexibilidade, mínima previsibilidade. Modelos pequenos entram em ping-pong. Bom para explorar, ruim para produção.

H Hierárquico

Supervisores de supervisores; cada subtime é um subgrafo compilado. É a estrutura do Agent Zero, com superior criando subordinado de contexto isolado. Use com mais de seis especialistas.

S Sequencial

A, depois B, depois C, sem decisão de roteamento. Se a ordem é sempre a mesma, NÃO use LLM para decidir o que já está decidido. Mais barato, mais rápido, mais confiável.

Swarm: enxame com estado compartilhado

1

É handoff generalizado

Qualquer agente pode passar o controle para qualquer outro, e o estado compartilhado registra quem está ativo.

2

O ganho é adaptabilidade

O caminho não é decidido de antemão: emerge da sequência de transferências que os agentes escolhem.

3

O custo é previsibilidade

O mesmo pedido pode percorrer caminhos diferentes em execuções diferentes. Testar fica mais difícil, e o traço fica mais longo.

4

Precisa de freio duplo

Contador de transferências e limite de tempo. Sem os dois, um enxame pode circular por horas com modelo local.

Escolhendo em cinco perguntas

Pergunta	Se sim	Por quê
A ordem é sempre a mesma?	Sequencial	Roteamento por LLM seria desperdício puro
Preciso de rastro claro de decisão?	Supervisor	Um único ponto de decisão, fácil de logar
O fluxo é conversa que muda de dono?	Handoff	Quem tem o contexto decide o próximo passo
São mais de seis especialistas?	Hierárquico	Um supervisor só não dá conta de escolher bem
Não sei ainda?	Supervisor	Padrão seguro; dá para migrar depois

O PONTO MAIS IMPORTANTE DO DIA

**Cada especialista recebe uma tarefa, não a conversa inteira.
E devolve um resultado, não o seu histórico.**

Isso é isolamento de contexto. Se você passar o histórico completo para cada especialista, o seu sistema multiagente é apenas um agente com prompt maior, e vai estourar o contexto no terceiro turno. Meça os tokens de entrada por agente e comprove.

Escolha a topologia

1

Extraí dados, validar, gerar PDF, enviar

Sempre nesta ordem.

2

Atendimento que às vezes precisa do time técnico e às vezes do financeiro

Depende do assunto.

3

Investigação de incidente, caminho imprevisível

Vários especialistas, ordem incerta.

03

Implementando

Command, handoff, especialistas e limite de ciclo.

A ferramenta de transferência

```
def criar_handoff(destino: str, quando: str):  
    @tool(f"transferir_para_{destino}", description=quando)  
    def ferramenta(  
        motivo: Annotated[str, "Por que esta transferencia e necessaria"],  
        state: Annotated[dict, InjectedState],  
        tool_call_id: Annotated[str, InjectedToolCallId],  
    ) -> Command:  
        recibo = ToolMessage(content=f"Transferido para {destino}. Motivo: {motivo}",  
                               name=f"transferir_para_{destino}",  
                               tool_call_id=tool_call_id)  
        return Command(goto=destino,  
                        graph=Command.PARENT,          # <- sem isto, o goto falha  
                        update={"messages": state["messages"] + [recibo],  
                               "agente_ativo": destino})  
    return ferramenta
```

graph igual a Command.PARENT sobe para o grafo pai. Sem isso, o goto procura o nó dentro do subgrafo do próprio agente e falha com um erro pouco claro.

Onde o handoff quebra

1 Faltou Command.PARENT

O goto procura o nó dentro do subgrafo do agente e não acha. O erro aponta para o lugar errado e a pessoa perde meia hora.

2 Faltou o ToolMessage de recibo

Toda tool call precisa de resposta. Sem o recibo, a API recusa o histórico na próxima chamada.

3 Description que diz o que faz

Transfere para o analista não diz nada. Transfira QUANDO a pergunta exigir conta ou comparação numérica diz tudo.

Duas implementações: meça as duas

A primeira é a que se ensina; a segunda costuma funcionar melhor com modelo local, porque tool calling é o que esses modelos foram treinados a fazer.

O campo motivo não é enfeite: é o log de decisão do sistema.

Exercício de quinze minutos: rode a mesma pergunta cinco vezes em cada versão e conte quantas vezes o roteamento fez sentido.

```
# A) structured output
class Roteamento(BaseModel):
    proximo: Literal["pesquisador", "analista",
                    "redator", "critico", "FIM"]

    instrucao: str
    motivo: str

d = llm.with_structured_output(Roteamento).invoke(msgs)

# B) ferramentas de handoff
supervisor = create_agent(
    model=llm,
    tools=[para_pesquisador, para_analista,
           para_redator, para_critico],
    system_prompt=PROMPT_SUPERVISOR)
```

A fábrica de nós especialistas

```
def fazer_especialista(nome: str, prompt: str, ferramentas: list, llm):  
    agente = create_agent(model=llm, tools=ferramentas, system_prompt=prompt)  
  
    def no(estado: EstadoEquipe) -> dict:  
        tarefa = (f"Tarefa: {estado['instrucao']}\n\n"          # <- so a instrucao  
                  f"Pergunta original: {estado['pergunta']}")  
        saida = agente.invoke({"messages": [HumanMessage(tarefa)]})  
        texto = saida["messages"][-1].content  
        return {"messages": [AIMessage(content=texto, name=nome)],  
                "achados": [f"[{nome}] {texto}"]}          # <- reducer operator.add  
    return no
```

O especialista é um agente completo por dentro, com o próprio loop e as próprias ferramentas. Por fora, é uma função que recebe tarefa e devolve resultado.

Dois papéis, escritos com cuidado

R Redator

- Escreve o relatório a partir dos achados
- Cita a fonte de cada afirmação numérica
- NÃO busca; usa apenas o que recebeu
- NÃO inventa número que não esteja nos achados
- Devolve markdown com no máximo 400 palavras

C Crítico

- Confere cada número contra a fonte citada
- Reprova se houver afirmação sem fonte
- Reprova se a pergunta original não foi respondida
- NÃO reescreve o texto; aponta o que corrigir
- Devolve veredito e lista numerada de problemas

Todo ciclo precisa de limite explícito

O crítico reprova, volta ao supervisor, que manda ao redator, que devolve ao crítico. Sem corte, isso roda para sempre, e com modelo local para sempre é literalmente a noite inteira.

Dois cortes, não um: o do supervisor e o do crítico. Ambos leem o mesmo contador no estado.

O contador vive no ESTADO, e não numa variável de loop: dá para inspecionar, persistir e mostrar na interface.

```
MAX_RODADAS = 3

def rotear_supervisor(estado) -> str:
    if estado.get("rodadas", 0) >= MAX_RODADAS:
        return END
    return (END if estado["proximo"] == "FIM"
            else estado["proximo"])

def rotear_critico(estado) -> str:
    if (estado["veredito"] == "aprovado"
        or estado.get("rodadas", 0) >= MAX_RODADAS):
        return END
    return "supervisor"
```

Laboratório da manhã: a equipe NEXUS

1

Escrever os quatro prompts primeiro

Pesquisador, Analista, Redator e Crítico. Cada um com o que fazer e o que NÃO fazer. Antes de qualquer código.

2

Montar a topologia supervisor

Com structured output no roteamento e a fábrica de especialistas.

3

Limitar o ciclo do Crítico

Contador no estado e corte nos dois roteadores. Provoque o limite de propósito.

4

Medir os tokens de entrada por agente

Se os quatro números forem parecidos, você não isolou o contexto. Conserte e meça de novo.

5

Extensão: a mesma equipe em handoff

Para quem terminar antes. É metade do entregável comparativo da tarde.

04

Paralelismo e subgrafos

Send, reducers e o bug mais difícil do dia.

Paralelismo não é especialização

1

Especialização divide POR COMPETÊNCIA

Papéis diferentes, prompts diferentes, ferramentas diferentes. Um pesquisa, outro escreve.

2

Paralelismo divide POR ITEM

O mesmo papel, trinta vezes, sobre trinta documentos independentes. Mesmo prompt, entradas diferentes.

3

As duas coisas se combinam

Um planejador quebra em subtemas, N pesquisadores idênticos rodam em paralelo, e um consolidador junta. É map-reduce.

Send: uma instância do nó por item

Quando há N itens independentes, executar em série é desperdício. Send cria uma instância do nó para cada item, com o seu próprio estado de entrada.

Os resultados voltam para o mesmo campo do estado, e é por isso que esse campo PRECISA de reducer de concatenação.

Sem o reducer, as execuções paralelas sobrescrevem umas às outras e você fica com o resultado de uma só, aleatoriamente.

```
from langgraph.types import Send

def distribuir(estado):
    return [Send("pesquisar_subtema",
                 {"subtema": s,
                  "pergunta": estado["pergunta"]})
            for s in estado["subtemas"]]

g.add_conditional_edges("planejador", distribuir,
                        ["pesquisar_subtema"])
g.add_edge("pesquisar_subtema", "consolidador")

# obrigatorio no estado:
achados: Annotated[list[str], operator.add]
```

O BUG MAIS DIFÍCIL DO DIA

**Sem reducer, três resultados paralelos viram um.
E funciona em trinta por cento das execuções.**

Três instâncias terminam quase ao mesmo tempo e escrevem no mesmo campo. Sem reducer, a última a escrever vence, e qual é a última muda a cada execução. O sistema parece funcionar, entrega resultado plausível, e está perdendo dois terços do trabalho.

Um grafo compilado pode ser nó de outro

A regra: as chaves de estado compartilhadas precisam ter o mesmo nome e o mesmo reducer nos dois grafos.

Se os esquemas forem diferentes, use um adaptador explícito. É mais verboso e muito mais fácil de depurar; prefira-o enquanto a turma estiver aprendendo.

```
# forma direta (mesmos nomes de chave)
time = construir_time_pesquisa().compile()
g_principal.add_node("time_pesquisa", time)

# forma com adaptador (recomendada em sala)
def no_time(estado: EstadoPai) -> dict:
    entrada = {"consulta": estado["pergunta"]}
    saida = time.invoke(entrada)
    return {"achados": saida["resultados"]}

g_principal.add_node("time_pesquisa", no_time)
```

O que declarar ao compor dois grafos

Item	Por que importa
Quais campos entram	Sem isso, o subgrafo lê campo que o pai não garante
Quais campos saem	O pai precisa saber o que esperar para rotear depois
Quais campos são compartilhados	Mesmo nome e mesmo reducer, ou o adaptador traduz
O que acontece em falha	Subgrafo que levanta exceção derruba o pai, salvo tratamento
Quem limita o ciclo	Se os dois têm ciclo, os dois precisam de contador

Três situações

1

Cinco resumos paralelos, e só um aparece no fim

O que faltou?

2

O consolidador rodou antes de todos terminarem

Isso é possível?

3

O subgrafo não acha um campo do estado

Onde olhar primeiro?

05

Context engineering

O recurso que ninguém orça, e que explica o multiagente.

A DEFINIÇÃO QUE VALE DECORAR

**Encontre o conjunto mínimo de tokens de alto sinal
que maximiza a probabilidade do resultado desejado.**

Contexto não é armazenamento, é orçamento. Cada token que você coloca disputa espaço com o trabalho de verdade. Multiagente existe, antes de tudo, por uma razão física: a janela é finita e um agente só não dá conta.

Por que mais contexto não é melhor contexto

1 Context rot

- Mais tokens, pior a recuperação de qualquer um
- Degradação contínua, não um degrau no limite
- Teto perceptível por volta de um milhão de tokens
- Janela de 200 mil não é convite a usar 200 mil

2 Context anxiety

- O modelo percebe a própria janela
- Encerra o trabalho antes da hora
- Estimativas muito precisas e erradas
- Sumiu sozinho na geração seguinte de modelos

Os quatro que aparecem em toda sala

Anti-padrão	O que é	Sintoma no NEXUS
Kitchen sink session	Tarefas sem relação na mesma sessão	Responde sobre o assunto anterior
Correções repetidas	Contexto poluído por tentativas falhas	Cada correção piora a resposta
Instruções longas demais	Arquivo de regras inflado	Ignora justamente as regras que importam
Exploração sem limite	Investigação irrestrita antes da tarefa	Estoura antes de começar o trabalho

Corrigiu duas vezes e ainda está errado? Limpe e recomece.

É contraintuitivo, porque a tentação é sempre explicar de novo. Mas cada correção fracassada fica no contexto, e o contexto poluído torna a próxima tentativa pior. Recomeçar com uma instrução melhor, escrita com o que você aprendeu, funciona melhor que insistir.

A diferença está no que se preserva

Estratégia	Mecanismo	Quando usar
Compaction	Resume o histórico e continua	Tarefa longa em que a continuidade importa
Context reset	Limpa e recomeça com estado estruturado	Quando a poluição é grave
Nova sessão	Conversa nova do zero	Troca de assunto

A técnica de compressão mais subestimada

A Anthropic descobriu que manter notas persistentes FORA do contexto é altamente eficaz. O Claude jogando Pokémon mantinha contagem de passos e, após um reset, lia as próprias notas e continuava sequências de horas.

No NEXUS isso já existe: rascunho e achados são notas fora da janela. O que muda é a intenção; deixam de ser detalhe de implementação e passam a ser a estratégia de compressão do sistema.

```
def no_pesquisador(estado: EstadoEquipe) -> dict:
    achados = buscar(estado["instrucao"])

    # a nota e curta e estruturada;
    # o texto bruto NAO volta ao contexto
    return {"achados": [
        f"[{a.metadata['source']}] "
        f"{resumir(a, limite=280)}"
        for a in achados]}

# o erro comum e o oposto: devolver ao
# supervisor todo o texto que o especialista
# leu. Funciona nas tres primeiras perguntas
# e estoura na quarta.
```

O orçamento de contexto do seu sistema

Componente	Tokens típicos	O seu
Prompt de sistema do especialista	200 a 600	
Descrições das ferramentas	50 por ferramenta	
Instrução da tarefa	50 a 200	
Achados recebidos	300 a 3000	
Histórico da conversa	0 a estourar	

Três decisões

1

A tarefa é longa e a continuidade importa

Compaction, reset ou sessão nova?

2

Você corrigiu o agente três vezes e piorou

Compaction, reset ou sessão nova?

3

O usuário mudou de assunto

Compaction, reset ou sessão nova?

06

Memória e interface

Lembrar o que importa, e deixar alguém usar isso.

O checkpointer resolve um problema e cria outro

Tipo	O que guarda	Onde vive	Quando é consultada
Curto prazo	As mensagens desta conversa	Checkpointer (thread_id)	Sempre, inteira
Semântica	Fatos duráveis sobre usuário e domínio	Vector store	Por similaridade com a pergunta
Episódica	Execuções passadas e resultados	Store ou banco	Quando o agente busca precedente

A mesma mecânica do RAG, aplicada a fatos

O LangGraph tem um Store separado do checkpointer, justamente para isso. Sem o índice de embeddings ele só busca por chave.

A busca funciona sem NENHUMA palavra em comum entre a consulta e o fato guardado. É a mesma propriedade que fez o RAG funcionar no dia 2.

```
from langgraph.store.memory import InMemoryStore
from langchain_ollama import OllamaEmbeddings

emb = OllamaEmbeddings(model="nomic-embed-text")

memoria = InMemoryStore(
    index={"embed": emb.embed_documents, "dims": 768})

memoria.put(
    ("usuario", "ana"),
    "preferencia-formato",
    {"texto": "Ana prefere relatorios curtos."})

# recupera por SIGNIFICADO, nao por chave
achados = memoria.search(
    ("usuario", "ana"),
    query="como devo formatar a resposta?",
    limit=3)

grafo = construtor.compile(
    checkpointer=saver, store=memoria)
```

O CUIDADO QUE NINGUÉM MENCIONA

Um agente que escreve na própria memória acumula lixo rápido.

Sem política de expiração ou deduplicação, em duas semanas o store tem quarenta variações de que o usuário gosta de respostas curtas. Em produção isso exige manutenção. No curso, basta o aluno saber que o problema existe, e um dos exercícios pede que ele meça quantas duplicatas o próprio agente criou.

Gradio: uma interface sem tocar no grafo

O NEXUS roda no terminal: ótimo para depurar, péssimo para demonstrar. O ponto de integração é um só: uma função que recebe mensagem e devolve resposta.

Use yield, e não return. Com return, tudo aparece de uma vez no fim e a demonstração perde o efeito.

```
import gradio as gr

def responder(mensagem, historico, thread_id):
    config = {"configurable":
              {"thread_id": thread_id or "demo"}}
    entrada = {"messages": [HumanMessage(mensagem)],
              "pergunta": mensagem,
              "rodadas": 0, "achados": []}

    parcial = ""
    for ev in equipe.stream(entrada, config,
                          stream_mode="updates"):
        for no, delta in ev.items():
            msgs = delta.get("messages", [])
            if msgs and msgs[-1].content:
                parcial += f"\n\n**[{no}]**\n{msgs[-1].content}"
            yield parcial

    with gr.Blocks(title="NEXUS") as app:
        thread = gr.Textbox(label="Sessao", value="demo")
        gr.ChatInterface(fn=responder,
                        additional_inputs=[thread],
                        type="messages")
```

Por que a interface vale o bloco de aula

T Terminal

- Você vê o traço completo
- Dá para inspecionar o estado
- Rápido de iterar
- Ninguém além de você consegue usar

G Interface

- A saída mostra qual nó falou
- O thread_id vira um campo na tela
- A arquitetura fica visível na demonstração
- Uma pessoa não programadora consegue testar

AVISO DE SEGURANÇA

share=True cria um túnel público.

Com um agente que lê disco e executa código, é péssima ideia.

É útil para mostrar ao cliente sem fazer deploy, e é exatamente por isso que as pessoas usam sem pensar. O agente do curso lê disco, escreve arquivos e executa código. No laboratório: sempre share=False, que é o padrão.

07

Observabilidade e fechamento

Com cinco agentes, `print()` não serve mais.

Quatro números que vale acompanhar

Métrica	O que revela	Sinal de alerta
Transições por execução	Supervisor indeciso, ping-pong entre agentes	Mais do que o dobro do esperado
Tokens por agente	Quem está carregando contexto demais	Um agente com cinco vezes os outros
Taxa de reprovação do crítico	Prompt do redator fraco ou critério vago	Acima de 50% ou abaixo de 5%
Tempo por nó	Onde vale paralelizar ou usar modelo menor	Um nó dominando o total

Uma linha por transição, em disco

```
for evento in equipe.stream(entrada, config, stream_mode="updates"):
    for no, delta in evento.items():
        msgs = delta.get("messages") or []
        texto = str(getattr(msgs[-1], "content", ""))[:400] if msgs else ""
        console.print(Panel(texto, title=no, border_style=CORES[no]))
        traco.write(json.dumps({"t": round(time.time() - t0, 2),
                                "no": no,
                                "chaves": list(delta.keys()),
                                "resumo": texto}, ensure_ascii=False) + "\n")
```

JSONL é o formato certo: uma linha por evento, fácil de filtrar com grep, fácil de carregar em pandas depois.

Três opções de rastreamento

1 LangSmith

Hospedado, integra sem mudar código, com árvore de chamadas e comparação entre execuções. Envia dados para fora da sua máquina.

2 Langfuse

Pode ser auto-hospedado, o que resolve o problema de privacidade. Instrumentação por callback, como vimos no dia 2.

3 OpenTelemetry

Se a sua empresa já tem observabilidade, o agente vira mais um serviço instrumentado, no mesmo painel do resto.

Laboratório da tarde: comparar para decidir

1

Instrumentar a equipe

Traço em JSONL com nó, tempo e chaves alteradas, e as quatro métricas calculadas ao fim.

2

Rodar a mesma bateria nas duas topologias

Cinco perguntas, supervisor e handoff. Mesmo modelo, mesmas ferramentas.

3

Preencher a tabela comparativa

Transições, tokens, tempo e se a resposta prestou. Sem a tabela, não há argumento.

4

Ligar o store de memória semântica

Guarde uma preferência do usuário e prove que ela é recuperada por significado.

5

Colocar a interface por cima

Vinte linhas, sem tocar no grafo. Demonstre com dois thread_id diferentes.

Erros clássicos do dia 4

Sintoma	Causa	Correção
Dois agentes se transferindo sem parar	Sem limite de rodadas	Contador no estado e corte nos dois roteadores
Contexto estoura no terceiro turno	Especialista recebendo o histórico inteiro	Passe só a instrução
Resultados paralelos somem	Falta reducer de concatenação	Annotated com operator.add
goto não encontra o nó	Falta graph igual a Command.PARENT	Adicione o parâmetro
Erro de tool call sem resposta	Handoff sem ToolMessage de recibo	Devolva o recibo sempre
Supervisor chama sempre o mesmo	Descrições parecidas demais	Diferencie por QUANDO usar
Crítico aprova qualquer coisa	Prompt genérico	Critérios verificáveis e numerados

As cinco camadas, e o que fizemos hoje

Camada	O que construímos hoje	Onde aprofunda
Instructions	Quatro prompts com o que fazer e o que NÃO fazer	Dia 5, prompt por modelo
Constraints	Limite de ciclo, orçamento de contexto por agente	Dia 5, no deploy
Feedback	O Crítico com critérios verificáveis e as quatro métricas	Dia 5, avaliação sistemática
Memory	Store semântico com namespace por usuário	Dia 5, no projeto final
Orchestration	Cinco topologias, paralelismo, subgrafos	Completa

PARA AMANHÃ

Estamos usando o mesmo modelo pequeno para tudo: supervisor, pesquisador, redator e crítico.

Faz sentido? O supervisor roda a cada transição e só precisa escolher entre cinco opções. O redator roda uma vez e precisa escrever bem. O crítico precisa ser rigoroso. E como se escolhe, entre os milhares de modelos do Hugging Face Hub, qual usar em cada papel?